

## Análise de Complexidade da USEI11 do projeto integrador

Esta secção apresenta a análise de complexidade das classes UpgradePlanner e orderAndCycles implementadas.

A análise segue a metodologia ESINF: primeiro contam-se as operações linha a linha quando aplicável, e depois apresenta-se a complexidade temporal total usando notação  $O(\cdot)$ .

Todo o código envolvido é determinístico, o que significa que o tempo de execução depende apenas do número de vértices  $V$  e arestas  $E$  do grafo, e não da ordem ou distribuição do input. Portanto, o caso esperado coincide com o pior caso, que é o caso relevante para esta análise.

Sendo:

- $V$  – número de vértices do grafo
- $E$  – número de arestas do grafo
- $d(v)$  – grau de saída do vértice  $v$  (número de vértices adjacentes)

### Classe UpgradePlanner

Método Upgrades(Graph<Node, Double> graph)

```
public List<Node> Upgrades(Graph<Node, Double> graph) {  
    List<Node> order = orderAndCycles.topologicalOrder(graph);  
    if (order != null){  
        return order;  
    }  
    return orderAndCycles.findFirstCycle(graph);  
}
```

Contagem de operações:

- Chamada a topologicalOrder(graph): ver análise na secção 2.1
- Comparação if (order != null): 1 comparação
- return order: 1 operação
- Chamada a findFirstCycle(graph): ver análise na secção 2.2

O método Upgrades chama primeiro topologicalOrder. Se o grafo for acíclico (DAG), este método retorna a ordem topológica e o algoritmo termina. Se o grafo contiver ciclos, topologicalOrder retorna null e então findFirstCycle é chamado para encontrar o primeiro ciclo.

No pior caso, ambos os métodos são executados sequencialmente quando existe um ciclo no grafo.

Sendo:

- Complexidade de topologicalOrder =  $V + E$  operações
- Complexidade de findFirstCycle =  $V + E$  operações

No pior caso (grafo com ciclos):

Total de operações =  $(V + E) + (V + E) = 2V + 2E$

Complexidade total:  $O(V + E)$

## Classe orderAndCycles

Método `topologicalOrder(Graph<Node, Double> graph)`

Este método implementa o algoritmo de Kahn para ordenação topológica, usando uma abordagem baseada em graus de entrada.

```
public static List<Node> topologicalOrder(Graph<Node, Double>
graph) {
    Map<Node, Integer> inDegree = new HashMap<>();

    for (Node v : graph.vertices()) {
        inDegree.put(v, graph.inDegree(v));
    }

    Queue<Node> queue = new LinkedList<>();
    for (Node v : inDegree.keySet()) {
        if (inDegree.get(v) == 0) {
            queue.add(v);
        }
    }

    List<Node> order = new ArrayList<>();

    while (!queue.isEmpty()) {
        Node v = queue.poll();
        order.add(v);

        for (Node adj : graph.adjVertices(v)) {
            inDegree.put(adj, inDegree.get(adj) - 1);
            if (inDegree.get(adj) == 0) {
                queue.add(adj);
            }
        }
    }

    if (order.size() != graph.numVertices()) {
        return null;
    }

    return order;
}
```

Criação do HashMap `inDegree`: 1 operação

Primeiro ciclo `for (Node v : graph.vertices())`:

- V iterações (uma por cada vértice)
- Cada iteração executa `inDegree.put()`: 1 operação
- Total: V operações

Criação da Queue: 1 operação

Segundo ciclo for (Node v : inDegree.keySet()):

- V iterações
- Cada iteração verifica o grau de entrada e possivelmente adiciona à fila: 1 operação
- Total: V operações

Criação da ArrayList order: 1 operação

Ciclo while (!queue.isEmpty()):

- Cada vértice é removido da fila exatamente uma vez: V iterações externas
- Para cada vértice v processado:
  - queue.poll(): 1 operação
  - order.add(v): 1 operação
  - Ciclo interno for (Node adj : graph.adjVertices(v)):
    - Itera sobre todos os adjacentes de v: d(v) iterações
    - Para cada adjacente: atualiza inDegree e possivelmente adiciona à queue: 1 operação por adjacente

A soma de todas as iterações do ciclo interno, para todos os vértices, é igual ao número total de arestas do grafo. Isto porque cada aresta (u,v) é examinada exatamente uma vez quando o vértice u é processado. Total do ciclo while: V (vértices processados) + E (arestas examinadas) operações

Verificação final order.size() != graph.numVertices(): 1 comparação

### **Explicação:**

O algoritmo de Kahn processa vértices sem dependências (grau de entrada 0), adiciona-os à ordem topológica e vai libertando os seus adjacentes. O ciclo while processa cada vértice exatamente uma vez e examina cada aresta exatamente uma vez, daí a complexidade linear.

### **Análise da complexidade:**

Total de operações:

- Inicialização do inDegree: V operações
- Preenchimento da queue inicial: V operações
- Processamento no while: V + E operações
- Verificação final: 1 operação

Total:  $V + V + V + E + 1 = 3V + E + 1$

**Complexidade total:  $O(V + E)$**

*Método findFirstCycle(Graph<Node, Double> graph)*

```
public static List<Node> findFirstCycle(Graph<Node, Double> graph) {
    Map<Node, Integer> state = new HashMap<>();
    Deque<Node> stack = new ArrayDeque<>();

    for (Node v : graph.vertices()) {
        state.put(v, 0);
    }

    for (Node v : graph.vertices()) {
        if (state.get(v) == 0) {
            List<Node> cycle = dfs(graph, v, state, stack);
            if (cycle != null) {
                return cycle;
            }
        }
    }

    return Collections.emptyList();
}
```

Criação do HashMap state e Deque stack: 2 operações

Primeiro ciclo for (Node v : graph.vertices()):

- V iterações
- Cada iteração executa state.put(v, 0): 1 operação
- Total: V operações

Segundo ciclo for (Node v : graph.vertices()):

- V iterações no ciclo externo
- Para cada vértice não visitado (state == 0), chama dfs()
- Embora o ciclo itere V vezes, cada vértice é visitado pela DFS apenas uma vez no total
- A DFS marca os vértices como visitados (state = 1 ou 2), evitando reprocessamento
- Complexidade total da DFS para todo o grafo: V + E operações (ver secção 2.3)

Return Collections.emptyList(): 1 operação

### **Explicação:**

O método inicializa todos os vértices com estado 0 (não visitado) e depois percorre-os, iniciando uma DFS para cada vértice ainda não visitado. A DFS usa três estados (0: não visitado, 1: em processamento, 2: completamente processado) e deteta ciclos quando encontra um adjacente com estado 1. Cada vértice é processado apenas uma vez pela DFS.

### **Análise da complexidade:**

Total de operações:

- Inicialização dos estados: V operações

- Verificações no ciclo externo: V operações
- Chamadas DFS (soma de todas): V + E operações

Total:  $V + V + (V + E) = 3V + E$

**Complexidade total:  $O(V + E)$**

Método *dfs*(Graph<Node, Double> graph, Node current, Map<Node, Integer> state, Deque<Node> stack)

```
private static List<Node> dfs(Graph<Node, Double> graph,
                             Node current,
                             Map<Node, Integer> state,
                             Deque<Node> stack) {

    state.put(current, 1);
    stack.push(current);

    for (Node adj : graph.adjVertices(current)) {
        if (state.get(adj) == 0) {
            List<Node> cycle = dfs(graph, adj, state, stack);
            if (cycle != null) return cycle;

        } else if (state.get(adj) == 1) {
            List<Node> cycle = new ArrayList<>();
            for (Node n : stack) {
                cycle.add(n);
                if (n.equals(adj)) break;
            }
            Collections.reverse(cycle);
            return cycle;
        }
    }

    stack.pop();
    state.put(current, 2);
    return null;
}
```

state.put(current, 1): 1 operação

stack.push(current): 1 operação

Ciclo for (Node adj : graph.adjVertices(current)):

- Para cada chamada dfs(v), o ciclo itera sobre todos os adjacentes de v: d(v) iterações
- Para cada adjacente:
  - state.get(adj): 1 operação

- Se estado == 0 (não visitado): chamada recursiva dfs()
- Se estado == 1 (em processamento - ciclo detetado):
  - Ciclo sobre a stack para reconstruir o ciclo: no máximo V iterações
  - cycle.add(n) para cada elemento: 1 operação por elemento
  - Collections.reverse(): no máximo V operações
  - Total quando ciclo é encontrado: no máximo 2V operações

stack.pop(): 1 operação

state.put(current, 2): 1 operação

#### Explicação:

A DFS visita cada vértice exatamente uma vez e examina cada aresta exatamente uma vez. Considerando todas as chamadas recursivas para o grafo completo:

- Cada vértice é visitado quando o seu estado muda de 0 para 1
- Durante cada visita, o algoritmo examina todas as arestas que partem desse vértice
- Somando para todos os vértices: V operações para vértices + E operações para arestas

#### Análise da complexidade:

Total de operações = V (visitar vértices) + E (examinar arestas)

**Complexidade total:  $O(V + E)$**

#### Tabela Resumo das Complexidades

| Método                            | Complexidade Temporal |
|-----------------------------------|-----------------------|
| UpgradePlanner.Upgrades()         | $O(V + E)$            |
| orderAndCycles.topologicalOrder() | $O(V + E)$            |
| orderAndCycles.findFirstCycle()   | $O(V + E)$            |
| orderAndCycles.dfs()              | $O(V + E)$            |